



Gymnázium J. K. Tyla

DarkFactory

Agentic and Harness Engineering: Artificial Intelligence in Practice

THESIS

Author: Patrik Marius, 4.D
Supervisor: Michal Dočekal

Extent: 1666 words / 11174 characters

Declaration

I declare that I prepared this specialized thesis independently under the supervision of the supervisor named on the title page. All sources used are listed in the bibliography and information derived from them is cited in the text. I agree that the printed version may be archived at Gymnázium J. K. Tyla and used there as a reference for future student work or for presenting education at GJKT.

In Hradci Králové on

.....

Author signature:

Annotation

[EN] This thesis examines the principles of agentic engineering: effective engineering practices for development with artificial intelligence through agentic systems and the architecture of these systems. The practical contribution of the thesis is the design and implementation of DarkFactory — an agentic harness installable as a GitHub App. The system aims to maximize automation of the development lifecycle, from interpreting requirements in GitHub Issues, through planning, to code development and merging changes. The thesis reflects that current agentic systems cannot be regarded as fully autonomous: language models require deterministic guardrails, context management, and human involvement (*Human-in-the-loop*).

Keywords

, Agent Harness, Agent Loop, , Embedding, , , Hook, Loop Engineering, Plugins, , Skills, , Transformer

Contents

Declaration	2
Annotation	3
Keywords	4
1 Introduction	7
1.1 Motivation and Problem Definition	7
1.1.1 Úvod	7
1.2 Thesis Objective and Research Questions	7
1.2.1 Úvod	7
1.2.2 Main Goal	7
1.2.2.1 Úvod	7
1.2.3 Sub-goals	7
1.2.3.1 Úvod	7
1.2.4 Research Questions	7
1.2.4.1 Úvod	7
1.3 Methodology	7
1.3.1 Úvod	8
2 Agentic AI	8
2.1 Development Environment and Practices	8
2.1.1 Úvod	8
2.1.2 Software Engineering	8
2.1.2.1 Úvod	8
2.1.3 Version control	8
2.1.3.1 Úvod	8
2.1.3.2 Git	8
2.1.3.2.1 Úvod	8
2.1.4 Continuous Integration	9
2.1.4.1 Úvod	9
2.2 Language Models, Chatbots, and Agents	9
2.2.1 Úvod	9
2.2.2 Large Language Model	10
2.2.2.1 Úvod	10
2.3 Agentic Engineering	10
2.3.1 Úvod	10
2.3.2 Prompt Engineering	10

2.3.2.1	Úvod	11
2.3.3	Agent Harness	11
2.3.3.1	Úvod	11
2.3.3.2	ReAct Loop	11
2.3.3.2.1	Úvod	11
2.3.3.3	Tool Calling	12
2.3.3.3.1	Úvod	12
2.3.3.4	Skills	12
2.3.3.4.1	Úvod	12
2.3.3.5	Context Engineering	13
2.3.3.5.1	Úvod	13
2.3.3.6	Workflow-graph Engineering	13
2.3.3.6.1	Úvod	13
3	DarkFactory: Harness Architecture - Practical Part	13
3.1	Development Environment and Practices	13
3.1.1	Úvod	13
3.1.2	Version control	13
3.1.2.1	Úvod	13
3.1.2.2	Git	13
3.1.2.2.1	Úvod	14
4	Results and Discussion	14
5	Conclusion	14
	List of figures and tables	15
	List of appendices	16
	References	17

1 Introduction

1.1 Motivation and Problem Definition

Ústřední inženýrská otázka této práce proto nespočívá v tom, zda jazykový model dokáže napsat fragment kódu. Zkoumáme, jaká kontrolní a dozorčí architektura — značovaná jako **Agent Harness** — musí model obklopovat, aby bylo možné jeho výstupům v produkčním repozitáři spolehlivě důvěřovat a dosáhnout vysoké míry autonomie se zachováním lidského dohledu.

1.1.1 Úvod

1.2 Thesis Objective and Research Questions

1.2.1 Úvod

1.2.2 Main Goal

Vymezit teoretické principy agentického inženýrství (*agentic engineering*) a navrhnout modulární architekturu agent harnessu pro automatizovaný vývoj softwaru se zachováním lidského dohledu v klíčových rozhodovacích bodech.

1.2.2.1 Úvod

1.2.3 Sub-goals

- Vymezit infrastrukturu pro správu verzí (Git, GitHub a kontinuální integraci).
- Analyzovat limity velkých jazykových modelů (dynamiku kontextového okna, jev Context Rot, ztrátovou kompresi a sémantický posun).
- Navrhnout architekturu agent harnessu zahrnující nástrojové smyčky (ReAct), bezpečnostní pískoviště a hierarchickou orchestraci subagentů.
- Formalizovat mechanismy zapojení člověka do smyčky (*Human-in-the-loop*), schvalovací brány a protokol revizních značek pro dohled nad textovými výstupy.

1.2.3.1 Úvod

1.2.4 Research Questions

1.2.4.1 Úvod

1.3 Methodology

Práce má teoreticko-architektonický a inženýrský charakter. Vzhledem k dynamickému vývoji v oblasti autonomního softwarového vývoje práce důsledně zachovává a integruje zavedené anglické odborné názvy (např. *harness*, *pull request*, *agent loop*, *prompt engineering*, *skills* či *context rot*). Použití této terminologie je integrální součástí práce, neboť tyto anglické pojmy představují de facto celosvětové průmyslové standardy (*industry standards*), jejichž doslovný český překlad by byl nejednoznačný, zavádějící či v rozporu s běžnou inženýrskou praxí.

Postup práce sleduje strukturu inženýrského cyklu:

- 1. Analýza konceptu: Systematické zmapování limitů autoregresivních modelů, dynamiky kontextového okna, jevu Context Rot a rozhraní nástrojů.
- 2. Návrh architektury: Formulace modulárního modelu agent harnessu, správy stavu, exekučního pískoviště, bezpečnostních pojistek a orchestrace subagentů.
- 3. Kritické zhodnocení: Porovnání navržených principů s volnými agentními smyčkami a vymezení provozních limitů autonomního inženýrství.

1.3.1 Úvod

2 Agentic AI

Úvod

2.1 Development Environment and Practices

The set of versioning, planning, integration, and verification practices that form a deterministic environment for agentic software development. ⁽¹⁾

2.1.1 Úvod

2.1.2 Software Engineering

The systematic application of engineering principles to the specification, design, implementation, verification, operation, and maintenance of software systems. ⁽¹⁾

2.1.2.1 Úvod

2.1.3 Version control

The management and tracking of changes to source files and other versioned artifacts so changes can be safely branched, merged, audited, and reverted when necessary. ⁽²⁾

2.1.3.1 Úvod

2.1.3.2 Git

A distributed version-control system that records project history, supports branching and merging, and enables deterministic return to earlier repository states. ⁽²⁾

2.1.3.2.1 Úvod

Pro autonomní vývoj softwaru je spolehlivá správa verzí naprosto nezbytným základem. Jazykové modely generují kód na základě statistické pravděpodobnosti, a proto se nevyhnutelně dopouštějí chyb, logických přehmatů či regresí. Verzovací systém vytváří bezpečné a deterministické prostředí, v němž lze každou úpravu zaznamenat, otestovat a v případě selhání kdykoliv vrátit zpět k funkčnímu stavu. Namísto teoretických abstrakcí práce přímo využívá distribuovaný systém `·` v kombinaci s platformou `·`.

Klíčové komponenty infrastruktury zahrnují:

- Distribuovaný systém Git ⁽²⁾: Ukládá kompletní historii projektu v podobě jednotlivých revizí (*commitů*). Vývojář i agent pracují s plnou lokální kopií repozitáře, což umožňuje provádět změny, přepínat větve a spouštět lokální testy zcela nezávisle na síťovém připojení.

- Platforma GitHub: Slouží jako centrální bod pro sdílení kódu, týmovou koordinaci a automatizaci:
 - Zadávání a sledování úkolů (Issues): Strukturovaná textová zadání požadavků a hlášení chyb, která agentovi slouží jako výchozí specifikace úlohy.
 - Revize změn (Pull Requests): Uživatelské rozhraní pro přehledné zobrazení diffu, diskusi nad kódem a formální schválení člověkem.
 - Automatizace (GitHub Actions): Běhové prostředí pro automatické spouštění testů, linterů a překladů při každé události v repozitáři.

2.1.4 Continuous Integration

A development practice in which changes are integrated frequently and automatically verified by builds, tests, and other checks so integration failures are detected early. ⁽³⁾

2.1.4.1 Úvod

2.2 Language Models, Chatbots, and Agents

The conceptual minimum about language models and their context needed to understand agentic systems. ⁽⁴⁾

2.2.1 Úvod

V agentickém softwarovém inženýrství vystupuje velký jazykový model (LLM) jako stochastické kognitivní jádro celého systému. Z hlediska vnitřní architektury se jedná o dekodérový transformer (*Decoder-only*; Transformer), jehož typickými představiteli jsou moderní modely řady Claude, GPT či DeepSeek ⁽⁴⁾. Role modelu nespočívá ve vystupování jako vševědoucí orákulum se spolehlivou znalostí okolního světa, nýbrž jako pokročilý generátor hypotéz, kódu a strukturovaných volání nástrojů řízený obdrženým kontextem.

Základní principy fungování modelu zahrnují:

- Autoregresivní predikce: Model zpracovává zadanou sekvenci textu a na jejím základě iterativně předpovídá nejpravděpodobnější následující symboly (tokeny).
- Stochastická povaha: Vzhledem k pravděpodobnostnímu vzorkování může model na totožný vstup reagovat mírně odlišně, což vyžaduje deterministické mantinely v nadřazeném agent harnessu.

Pro efektivní nasazení modelu do vývojového cyklu je nezbytné porozumět způsobu, jakým reprezentuje informace a jaké fyzické limity vymezují jeho operační paměť. • — [CZ] Systém založený na jazykovém modelu určený primárně k textové interakci s uživatelem; odpovídá na jednotlivé požadavky, ale sám o sobě nedisponuje autonomní prováděcí smyčkou ani nástroji pro samostatnou modifikaci okolního prostředí.. • — [CZ] Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí vícezkrokové akce směřující k dosažení zadaného inženýrského cíle.. Rozdíl mezi nimi nespočívá v odlišném jazykovém modelu, ale v architektuře jeho zapojení do pracovního prostředí.

Srovnání obou přístupů:

- Konverzační chatbot:
 - Reaguje pouze na přímé textové výzvy v uzavřeném okně chatu.
 - Nemá přímý přístup k souborovému systému ani k nástrojům operačního systému.
 - Uživatel musí navržený kód ručně zkopírovat, vložit do projektu a otestovat.
- Autonomní agent:
 - Je vybaven sadou výkonných nástrojů (*tools*) pro práci s repozitářem.
 - Aktivně prozkoumává soubory, modifikuje zdrojový kód, spouští testy a interpretuje jejich návratové kódy.
 - Funguje v autonomní prováděcí smyčce, v níž iterativně reaguje na reálnou odezvu vývojového prostředí.

2.2.2 Large Language Model

Velký jazykový model je neuronový model trénovaný nad rozsáhlými textovými daty, který autoregresivně zpracovává a generuje posloupnosti tokenů.

2.2.2.1 Úvod

Jazykový model nepracuje přímo se znaky ani slovy v lidském slova smyslu. Vstupní text je nejprve deterministickým algoritmem převeden na číselné reprezentace, se kterými následně počítají maticové vrstvy neuronové sítě.

Tento proces zahrnuje následující pojmy:

- Tokeny a tokenizér (): Token představuje základní diskrétní jednotku (celé slovo, slabiku či fragment znaků). Převod mezi textem a posloupností číselných tokenů zajišťuje tokenizér (nejčastěji na bázi algoritmu Byte Pair Encoding, BPE).
- **Embedding** — [cz] Vícerozměrná vektorová reprezentace tokenů nebo jiných dat, v níž numerické vztahy mezi vektory zachycují užitečné sémantické vztahy mezi reprezentacemi. ⁽⁵⁾ (např. vektorová analogie král — muž + žena $\approx$ královna).
- Jazyková asymetrie tokenizace: Vzhledem k trénovacím datům optimalizovaným primárně pro angličtinu spotřebovávají flektivní jazyky s bohatou diakritikou (včetně češtiny) 2× až 3× více tokenů pro vyjádření téhož významu.

Z inženýrského hlediska je proto žádoucí vést systémové prompty, technické plány i komunikaci mezi nástroji v angličtině, aby se šetřila kapacita kontextu a snížila latence inference.

2.3 Agentic Engineering

An engineering discipline focused on designing, orchestrating, and operating agentic systems around language models, including tools, context, execution loops, guardrails, and human oversight. ⁽⁶⁾

2.3.1 Úvod

2.3.2 Prompt Engineering

An engineering discipline for systematically designing, structuring, and optimizing instructions and system prompts to guide and constrain language-model behavior. ⁽⁷⁾

2.3.2.1 Úvod

2.3.3 Agent Harness

Agent harness — an application and orchestration layer surrounding a model's inference core that provides the tool runtime, dynamic context-window management, guardrails, state handling, and deterministic control over the request lifecycle. ⁽⁸⁾

2.3.3.1 Úvod

Samotné inferenční jádro provádí výhradně matematické maticové operace nad zadanými váhami a vektory tokenů; orchestraci, práci se soubory, správu stavu a bezpečnostní mantinely zajišťuje Agent Harness .

Ústřední prováděcí funkce, která v architektuře harnessu řídí iterativní koordinaci agenta v reálném vývojovém prostředí, je **Agent Loop** . [cz] Iterativní prováděcí cyklus autonomního agenta založený na vzoru ReAct (Reasoning + Acting), v němž model střídavě uvažuje, volá nástroje a vyhodnocuje pozorování z běhového prostředí..

2.3.3.2 ReAct Loop

An iterative execution cycle of an autonomous agent based on the ReAct pattern (Reasoning + Acting), in which the model alternates between reasoning, tool calls, and evaluation of observations from the runtime environment. ⁽⁹⁾

2.3.3.2.1 Úvod

Agentní smyčka (*Agent Loop*) představuje výkonné jádro celého agent harnessu. Zatímco pasivní konverzační chatbot jednorázově odpoví na uživatelský dotaz a čeká na další vstup, agentní smyčka autonomně udržuje kontinuální iterativní proces, v němž harness opakovaně vyhodnocuje stav repozitáře, volá jazykový model a vykonává požadované systémové akce.

V každé iteraci agentní smyčky harness zajišťuje tyto klíčové funkce:

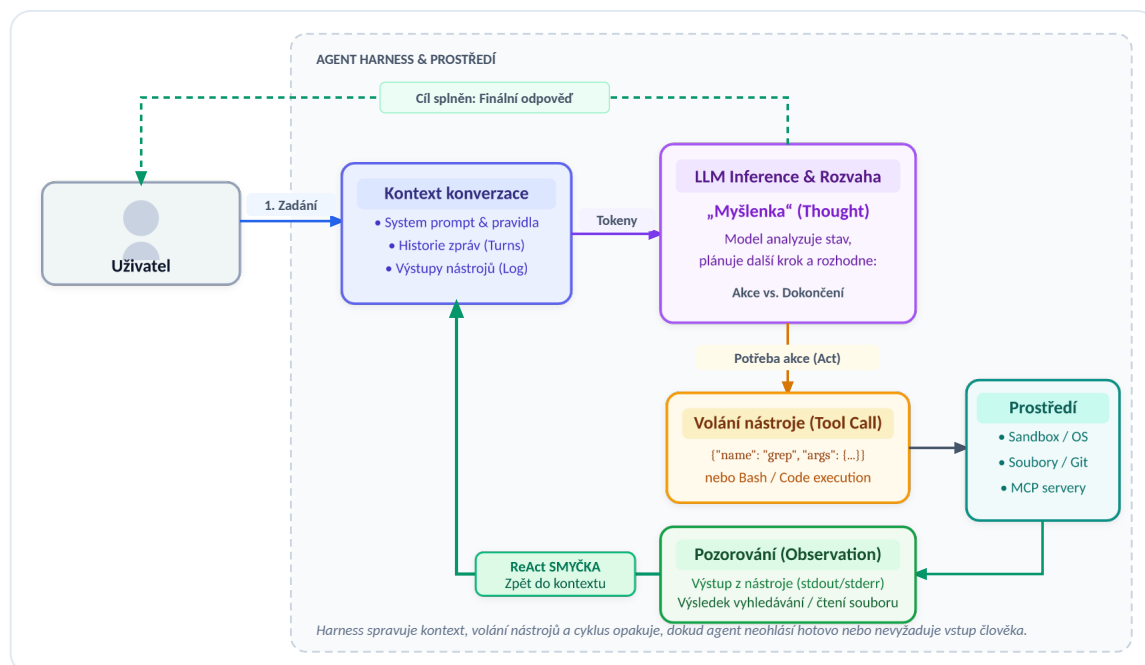
- Inicializace a správa sezení: Sestavení systémového promptu, dynamická injekce kontextu repozitáře a sledování spotřeby tokenů.
- Běhové prostředí nástrojů: Bezpečné spouštění příkazů v operačním systému a zpětné předávání výstupů modelu.
- Řízení stavových přechodů a vynucování mantinelů (Loop Engineering): Dohled nad dodržováním procesních pravidel, detekce a zastavení uvíznutých běhů a vynucování lidských schvalovacích bran.

Vnitřní kognitivní krok modelu uvnitř smyčky se řídí operačním vzorem ReAct (*Reasoning + Acting*) ⁽⁹⁾, který propojuje rozvahu s přímým jednáním. Tento prováděcí cyklus sestává ze čtyř navazujících fází znázorněných na Obrázek 1:

1. Rozvaha (*Thought*): Model vyhodnotí aktuální stav kontextu a formuluje svůj nejbližší záměr.
2. Volání nástroje (*Tool Call*): Emitování strukturovaného požadavku na provedení konkrétní akce s určenými parametry.
3. Vykonání a pozorování (*Observation*): Harness bezpečně provede akci v systému a výstup (výpis souboru či chybovou zprávu) vloží zpět do kontextu.

4. Navazující iterace: Model v dalším tahu analyzuje získanou odezvu a rozhoduje o dalším kroku.

Kvalita a provozní spolehlivost celého systému tak závisí v první řadě na robustnosti architektury harnessu a spolehlivosti jeho agentní smyčky, nikoliv pouze na samotném jazykovém modelu.



Obrázek 1: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

2.3.3.3 Tool Calling

A mechanism through which a model structurally asks a harness to execute an external action or function with validated parameters. ⁽¹⁰⁾

2.3.3.3.1 Úvod

2.3.3.4 Skills

Reusable modular packages of instructions (typically defined in a SKILL.md file), procedural rules, and optional helper scripts or resources that a harness dynamically loads into an agent's context for a particular class of task. ⁽¹¹⁾

2.3.3.4.1 Úvod

Se vzrůstající komplexitou úloh nelze veškeré instrukce, skripty a doménové znalosti vkládat do základního systémového promptu. K modulárnímu rozšíření schopností agenta slouží **Skills**.

Architektura dovedností staví na následujících principech:

- **Definiční soubor SKILL.md:** Dovednost tvoří adresář obsahující definiční soubor se strukturovanou hlavičkou (YAML frontmatter vymezující název a popis role) a detailním návodem k použití.

- Dynamické načítání pro úsporu kontextu: Do výchozího promptu se vloží pouze stručný přehled dostupných dovedností. Kompletní instrukce a skripty se do kontextu načtou až v okamžiku, kdy agent danou dovednost explicitně vyvolá.
- Skripty () a záchytné body (Hook): Dovednosti mohou obsahovat deterministické skripty pro rutinní transformace kódu a událostní háčky vyvolávané při stavových přechodech harnessu.

Kromě kontextových dovedností využívají pokročilé řídicí architektury také programové **Plugins** — [cz] Rozšíření běžící přímo v prostředí harnessu, která rozšiřují jeho exekuční jádro o specializované systémové adaptéry, ovladače nástrojů a deterministické záchytné body.. Zatímco *Skills* fungují jako kontextové procedury a instrukce interpretované modelem, pluginy rozšiřují samotný harness na nativní systémové úrovni.

2.3.3.5 Context Engineering

The systematic design, selection, ordering, and lifecycle management of information made available to a model in active context, including instructions, memory, tool results, and externally retrieved data. ⁽¹²⁾

2.3.3.5.1 Úvod

2.3.3.6 Workflow-graph Engineering

The design of agentic or automation workflows as explicit graphs of nodes, dependencies, and transitions rather than as one unconstrained loop. ⁽¹³⁾

2.3.3.6.1 Úvod

Monolitická agentní smyčka selhává při řešení komplexních, vícefázových úloh. Pro spolehlivé škálování se v moderních systémech uplatňuje hierarchická dělba práce a formalizace procesu do podoby grafu.

3 DarkFactory: Harness Architecture - Practical Part

Úvod

3.1 Development Environment and Practices

The set of versioning, planning, integration, and verification practices that form a deterministic environment for agentic software development. ⁽¹⁾

3.1.1 Úvod

3.1.2 Version control

The management and tracking of changes to source files and other versioned artifacts so changes can be safely branched, merged, audited, and reverted when necessary. ⁽²⁾

3.1.2.1 Úvod

3.1.2.2 Git

A distributed version-control system that records project history, supports branching and merging, and enables deterministic return to earlier repository states. ⁽²⁾

3.1.2.2.1 Úvod

4 Results and Discussion

5 Conclusion

List of figures and tables

Obrázek 1	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.	12
-----------	--	----

List of appendices

References

1. SOMMERVILLE, Ian. *Software Engineering*. 10. Pearson, 2016. ISBN 978-0-13-394303-0.
2. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
3. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
4. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
5. MIKOLOV, Tomas, CHEN, Kai, CORRADO, Greg a DEAN, Jeffrey. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*. Online. 2013. Available from: <https://arxiv.org/abs/1301.3781>
6. WANG, Lei, MA, Chen, FENG, Xueyang, ZHANG, Zeyu, YANG, Hao, ZHANG, Jingsen, CHEN, Zhiyuan, TANG, Jiakai, CHEN, Xu, LIN, Yankai, ZHAO, Wayne Xin, WEI, Zhewei a WEN, Ji-Rong. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science*. Online. 2024. Vol. 18, no. 6, p. 186345. Available from: <https://arxiv.org/abs/2308.11432>
7. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
8. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
9. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
10. SCHICK, Timo, DWIVEDI-YU, Jane, DESS\I, Roberto, RAILEANU, Roberta, LOMELI, Maria, ZETTLEMOYER, Luke, CANCEDDA, Nicola a SCIALOM, Thomas. Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 68539–68551. Available from: <https://arxiv.org/abs/2302.04761>
11. ANTHROPIC. Building effective agents. Online. 2024. Available from: <https://www.anthropic.com/research/building-effective-agents>
12. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>

13. WU, Qingyun, BANSAL, Gagan, ZHANG, Jieyu, WU, Yiran, LI, Beibin, ZHU, Erkang, JIANG, Li, ZHANG, Xiaoyun, ZHANG, Chi, LIU, Jue, AWADALLAH, Ahmed Hassan, WHITE, Ryan W., BURGER, Doug a WANG, Chi. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*. Online. 2023. Available from: <https://arxiv.org/abs/2308.08155>
14. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
15. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
16. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
17. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>
18. WOOLDRIDGE, Michael a JENNINGS, Nicholas R. Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review*. 1995. Vol. 10, no. 2, p. 115–152.
19. LEWIS, Patrick, PEREZ, Ethan, PIKTUS, Aleksandra, PETRONI, Fabio, KARPUKHIN, Vladimir, GOYAL, Naman, KÜTTLER, Heinrich, LEWIS, Mike, YIH, Wen-tau, ROCKTÄSCHEL, Tim, RIEDEL, Sebastian a KIELA, Douwe. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33, p. 9459–9474. Available from: <https://arxiv.org/abs/2005.11401>
20. JIANG, Huiqiang, WU, Qianhui, LIN, Chin-Yew, YANG, Yuqing a QIU, Lili. LLMingua: Compressing Context for Accelerated Inference of Large Language Models. *arXiv preprint arXiv:2310.05736*. Online. 2023. Available from: <https://arxiv.org/abs/2310.05736>
21. SHINN, Noah, CASSANO, Federico, GOPINATH, Ashwin, NARASIMHAN, Karthik a YAO, Shunyu. Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 8634–8652. Available from: <https://arxiv.org/abs/2303.11366>
22. AGACHE, Alexandru, DEACONESCU, Razvan, IORDACHE, Mihai, LUPU, Alexandra, RADU, Vlad, BUGA, Cezar, BALAN, Catalin a FLORESCU, Andreea. Firecracker: Lightweight Virtualization for Serverless Applications. In : *17th USENIX Symposium on Networked*

- Systems Design and Implementation (NSDI 20)*. Online. 2020. p. 419–434. Available from: <https://www.usenix.org/conference/nsdi20/presentation/agache>
23. MOSQUEIRA-REY, Eduardo, HERNÁNDEZ-PEREIRA, Elena, ALONSO-RÍOS, David, BOBES-BASCARÁN, José a FERNÁNDEZ-LEAL, Ángel. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*. Online. 2023. Vol. 56, no. 4, p. 3005–3054. Available from: <https://doi.org/10.1007/s10462-022-10246-w>
 24. SENNRICH, Rico, HADDOW, Barry a BIRCH, Alexandra. Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Online. 2016. P. 1715–1725. Available from: <https://aclanthology.org/P16-1162/>
 25. MERKEL, Dirk. Docker: lightweight linux containers for consistent development and deployment. *Linux Journal*. 2014. Vol. 2014, no. 239, p. 2.
 26. KINSMAN, Timothy, WESSEL, Mairieli, GEROSA, Marco A. a TREUDE, Christoph. How do software developers use GitHub Actions to automate their workflows?. In : *Proceedings of the 18th International Conference on Mining Software Repositories*. 2021. p. 420–431.
 27. DABBISH, Laura, STUART, Colleen, TSAY, Jason a HERBSLEB, Jim. Social coding in GitHub: transparency and collaboration in an open software repository. In : *Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work*. 2012. p. 1277–1286.